Lab 2

Introduction:

This report outlines the steps taken to set up and configure a Hadoop cluster consisting of a master node and a worker node. Additionally, it details the development and execution of MapReduce programs for analyzing large log files, as per the Lab 2 assignment.

Initially, we created two instances based on the prebuilt image: **"E516-Hadoop-Image-V4"**

- **node-master** (Master Node)
- **node-worker1** (Worker Node)

Configuration:

1. Updating the Hosts File
   To ensure proper name resolution between instances, we edited the **/etc/hosts** file on both instances by adding the private IP addresses of all node

```
  GNU nano 6.2                                hosts
127.0.0.1 localhost
10.3.34.34 node-master
10.3.34.242 node-worker1

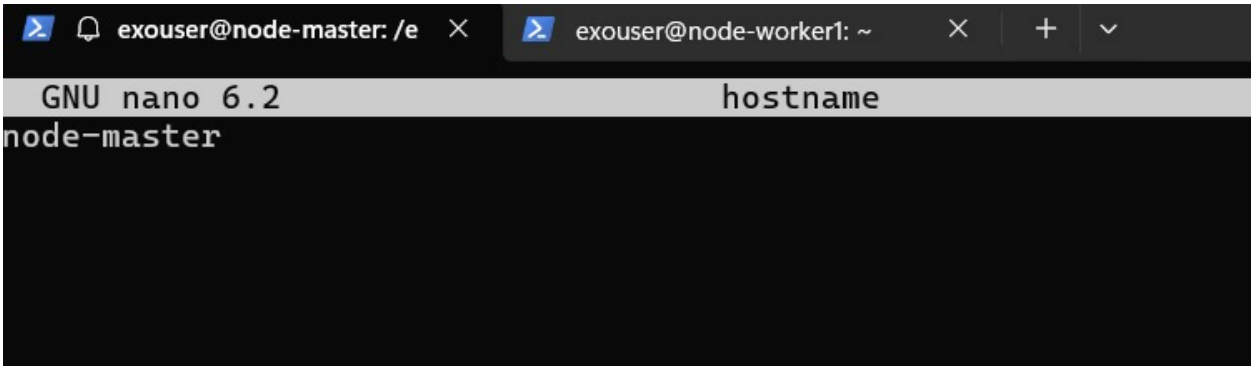
# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
ff02::3 ip6-allhosts
```

```
exouser@node-worker1: /etc
  GNU nano 6.2                                hosts
127.0.0.1 localhost
10.3.34.34 node-master
10.3.34.242 node-worker1

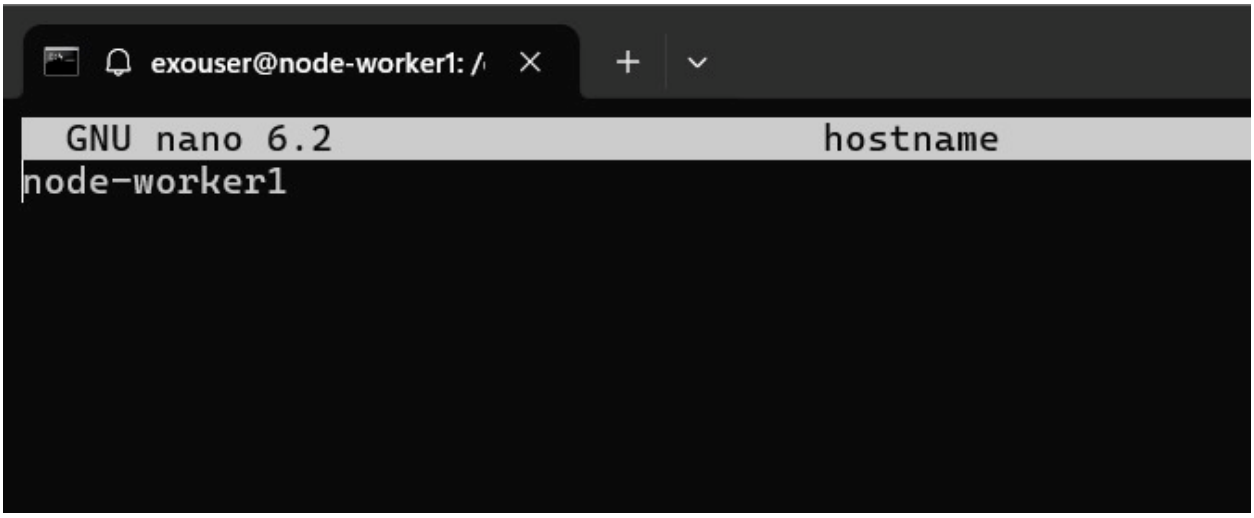
# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
ff02::3 ip6-allhosts
```

2. We checked the **/etc/hostname** file on each instance to confirm that it correctly reflected the instance's name.

   node-master instance:



   node-worker1 instance:



3. Configuring firewall rules: -
   To allow communication between nodes, we executed the following command on each instance:

```
exouser@node-master:/$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp (OpenSSH)           ALLOW IN    Anywhere
22/tcp                     ALLOW IN    Anywhere
49528/tcp                  ALLOW IN    Anywhere
Anywhere                   ALLOW IN    68.50.33.45
Anywhere                   ALLOW IN    10.3.34.34
Anywhere                   ALLOW IN    10.3.34.242
22/tcp (OpenSSH (v6))      ALLOW IN    Anywhere (v6)
22/tcp (v6)                ALLOW IN    Anywhere (v6)
49528/tcp (v6)             ALLOW IN    Anywhere (v6)
```

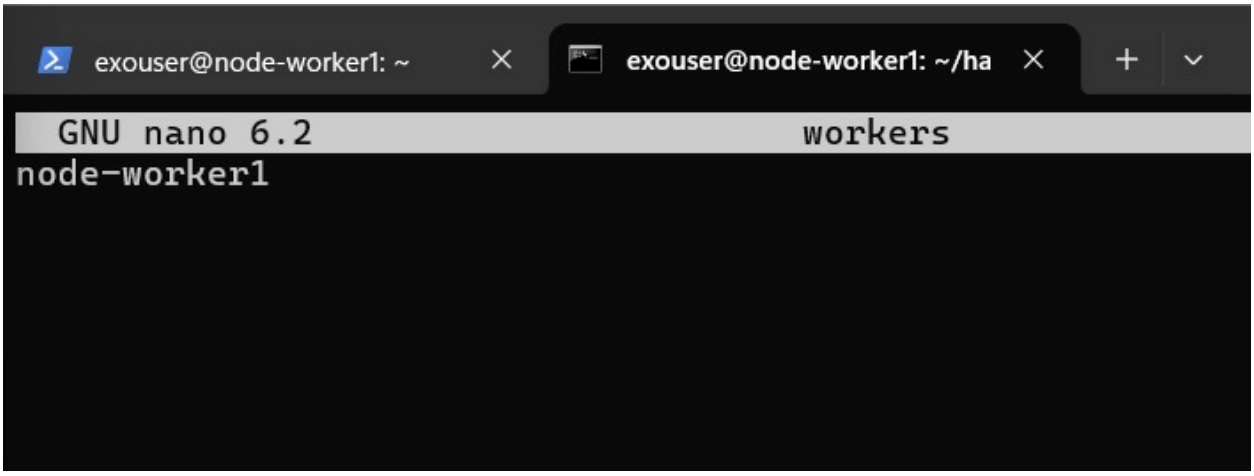
```
exouser@node-worker1:/$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp (OpenSSH)           ALLOW IN    Anywhere
22/tcp                     ALLOW IN    Anywhere
49528/tcp                  ALLOW IN    Anywhere
Anywhere                   ALLOW IN    10.3.34.242
Anywhere                   ALLOW IN    10.3.34.0/24
Anywhere                   ALLOW IN    10.3.34.34
22/tcp (OpenSSH (v6))      ALLOW IN    Anywhere (v6)
22/tcp (v6)                ALLOW IN    Anywhere (v6)
49528/tcp (v6)             ALLOW IN    Anywhere (v6)

exouser@node-worker1:/$
```

This ensured that necessary connections could be established between the instances.

We modified the worker node configuration in the hadoop workers file located at **hadoop-3.4.0/etc/hadoop/workers**



To allow monitoring of HDFS and Yarn services from a web browser, we opened the necessary ports only on the master node by executing the following command: $sudo ufw allow from [Your Laptop IP Address]

- HDFS web UI: accessible on port 9870
- Yarn resource manager UI: accessible on port 8088

Developing and running MapReduce programs: -

We checked If all the Hadoop services were running in both Master and Worker nodes using the command *jps*

```
exouser@node-master:~$ jps
827085 NodeManager
850941 SecondaryNameNode
1673 Bootstrap
851174 ResourceManager
98139 DataNode
891510 Jps
850691 NameNode
```

```
exouser@node-worker1:~/hadoop-3.4.0/etc/hadoop$ jps
228546 NodeManager
228416 DataNode
1700 Bootstrap
254281 Jps
208584 SecondaryNameNode
```

As part of the assignment, we implemented MapReduce programs to analyze a large log file (sample.log). The task was divided into two parts:

Part 1: output the top k IP addresses for each hour

- We analyzed the provided stat1 and stat2 examples written in python and java.
- We implemented our own MapReduce program based on these examples.
- The program processes log data, extracts timestamps, and counts occurrences of IP addresses per hour.
- The Reduce function aggregates and sorts the top k IP addresses for each hour.

Stat1 program:

```
  GNU nano 6.2                    map_stage_stat1.py

import re
import sys


pat = re.compile(r'(?P<ip>\d+\.\d+\.\d+\.\d+).*?\[(?P<date_time>[^\]]+)\]')


for line in sys.stdin:
    match = pat.search(line)
    if match:
        ip = match.group('ip')
        date_time = match.group('date_time')


        hour = date_time.split(":")[1]


        print(f"{hour}\t{ip}\t1")
```

```
exouser@node-master:~$ hadoop jar /home/exouser/hadoop-3.4.0/share/hadoop/tools/lib/hadoop-streaming-3.4.0.jar \
  -D mapred.reduce.tasks=1 \
  -input /input/sample.log \
  -output /output_stat1 \
  -mapper "python3 map_stage_stat1.py 5" \
  -reducer "python3 reduce_stage_stat1_stat2.py 5" \
  -file /home/exouser/map_stage_stat1.py \
  -file /home/exouser/reduce_stage_stat1_stat2.py
2025-03-02 17:13:53,433 WARN streaming.StreamJob: -file option is deprecated, please use generic option -files instead.
packageJobJar: [/home/exouser/map_stage_stat1.py, /home/exouser/reduce_stage_stat1_stat2.py, /tmp/hadoop-unjar6098067862091775 8/] [] /tmp/streamjob1769336939738979 7270.jar tmpDir=null
2025-03-02 17:13:54,130 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at node-master/10.3.34.34:8032
2025-03-02 17:13:54,263 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at node-master/10.3.34.34:8032
2025-03-02 17:13:54,478 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/exouser/.staging/job_1740948802585_0005
2025-03-02 17:13:54,825 INFO mapred.FileInputFormat: Total input files to process : 1
2025-03-02 17:13:54,903 INFO mapreduce.JobSubmitter: number of splits:2
2025-03-02 17:13:54,945 INFO Configuration.deprecation: mapred.reduce.tasks is deprecated. Instead, use mapreduce.job.reduces
2025-03-02 17:13:55,081 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1740948802585_0005
2025-03-02 17:13:55,082 INFO mapreduce.JobSubmitter: Executing with tokens: []
2025-03-02 17:13:55,234 INFO conf.Configuration: resource-types.xml not found
2025-03-02 17:13:55,234 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2025-03-02 17:13:55,286 INFO impl.YarnClientImpl: Submitted application application_1740948802585_0005
2025-03-02 17:13:55,308 INFO mapreduce.Job: The url to track the job: http://node-master:8088/proxy/application_1740948802585_0005/
2025-03-02 17:13:55,309 INFO mapreduce.Job: Running job: job_1740948802585_0005
2025-03-02 17:14:00,434 INFO mapreduce.Job: Job job_1740948802585_0005 running in uber mode : false
2025-03-02 17:14:00,435 INFO mapreduce.Job:  map 0% reduce 0%
2025-03-02 17:14:05,501 INFO mapreduce.Job:  map 100% reduce 0%
2025-03-02 17:14:09,526 INFO mapreduce.Job:  map 100% reduce 100%
2025-03-02 17:14:10,539 INFO mapreduce.Job: Job job_1740948802585_0005 completed successfully
2025-03-02 17:14:10,624 INFO mapreduce.Job: Counters: 55
```

Output:

```
exouser@node-master:~$ hdfs dfs -cat /output_stat1/part-00000
66.111.54.249   38
5.211.97.39     36
66.249.66.194   31
31.56.96.51     22
5.209.200.218   21
exouser@node-master:~$
```

Part 2: implementing parallel search with time period filtering

- We extended our program to accept a time period parameter from the user.
- The program filters records within the specified time range (e.g., 00:00 to 01:00).
- The output lists the top k IPs within the given time frame.

Stat 2 program:

exouser@node-master: ~

```
 GNU nano 6.2                    map_stage_stat2.py *
import re
import sys

time_period = sys.argv[1]
start_hour, end_hour = map(int, time_period.split('-'))

pat = re.compile(r'(?P<ip>\d+\.\d+\.\d+\.\d+).?\d{4}:(?P<hour>\d{2}):\d{2}.?')

for line in sys.stdin:
    match = pat.search(line)
    if match:
        hour = int(match.group('hour'))
        ip = match.group('ip')

        if start_hour <= hour < end_hour:
            print(f"{hour}\t{ip}\t1")
```

Because the Map stage has already structured the intermediate data at an hourly level, stat2's Reduce stage can use the same logic as stat1 without any additional modifications. This ensures efficiency by avoiding unnecessary re-grouping or processing

```
import sys
from collections import defaultdict
from operator import itemgetter

top_k_flag = False
K = 10

if len(sys.argv) > 1:
    if sys.argv[1].isdigit():
        K = int(sys.argv[1])
        top_k_flag = True
    else:
        time_period = sys.argv[1]

hourly_ip_count = defaultdict(lambda: defaultdict(int))

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue

    parts = line.split('\t')

    if len(parts) != 3:
        continue

    key, ip, count = parts

    try:
        count = int(count)
        if top_k_flag:
            hourly_ip_count[key][ip] += count
        else:
            hourly_ip_count[key][ip] += count
    except ValueError:
        continue

if top_k_flag:
    for hour, ip_count in hourly_ip_count.items():
        sorted_ips = sorted(ip_count.items(), key=itemgetter(1), reverse=True)[:K]

        for ip, count in sorted_ips:
            print(f"{ip}\t{count}")

else:
    for hour, ip_count in hourly_ip_count.items():
        sorted_ips = sorted(ip_count.items(), key=itemgetter(0), reverse=True)[:K]

        for ip, count in sorted_ips:
            print(f"{hour}\t{ip}\t{count}")
```

```
exouser@node-master:~$ hadoop jar /home/exouser/hadoop-3.4.0/share/hadoop/tools/lib/hadoop-streaming-3.4.0.jar  -D mapred.reduce.tasks=1  -input /input/sample.log  -ou
tput /output_stat2  -mapper "python3 map_stage_stat2.py 03-04"  -reducer "python3 reduce_stage_stat1_stat2.py 03-04 5"  -file /home/exouser/map_stage_stat2.py  -file
/home/exouser/reduce_stage_stat1_stat2.py
2025-03-02 16:44:41,359 WARN streaming.StreamJob: -file option is deprecated, please use generic option -files instead.
packageJobJar: [/home/exouser/map_stage_stat2.py, /home/exouser/reduce_stage_stat1_stat2.py, /tmp/hadoop-unjar5794472348235367700/] [] /tmp/streamjob7073500066920172066.j
ar tmpDir=null
2025-03-02 16:44:42,117 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at node-master/10.3.34.34:8032
2025-03-02 16:44:42,282 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at node-master/10.3.34.34:8032
2025-03-02 16:44:42,498 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/exouser/.staging/job_1740948802585_0003
2025-03-02 16:44:42,910 INFO mapred.FileInputFormat: Total input files to process : 1
2025-03-02 16:44:42,977 INFO mapreduce.JobSubmitter: number of splits:2
2025-03-02 16:44:43,025 INFO Configuration.deprecation: mapred.reduce.tasks is deprecated. Instead, use mapreduce.job.reduces
2025-03-02 16:44:43,567 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1740948802585_0003
2025-03-02 16:44:43,567 INFO mapreduce.JobSubmitter: Executing with tokens: []
2025-03-02 16:44:43,714 INFO conf.Configuration: resource-types.xml not found
2025-03-02 16:44:43,714 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2025-03-02 16:44:43,771 INFO impl.YarnClientImpl: Submitted application application_1740948802585_0003
2025-03-02 16:44:43,796 INFO mapreduce.Job: The url to track the job: http://node-master:8088/proxy/application_1740948802585_0003/
2025-03-02 16:44:43,798 INFO mapreduce.Job: Running job: job_1740948802585_0003
2025-03-02 16:44:48,916 INFO mapreduce.Job: Job job_1740948802585_0003 running in uber mode : false
2025-03-02 16:44:48,917 INFO mapreduce.Job:  map 0% reduce 0%
2025-03-02 16:44:53,992 INFO mapreduce.Job:  map 100% reduce 0%
2025-03-02 16:44:58,022 INFO mapreduce.Job:  map 100% reduce 100%
2025-03-02 16:44:59,042 INFO mapreduce.Job: Job job_1740948802585_0003 completed successfully
2025-03-02 16:44:59,139 INFO mapreduce.Job: Counters: 55
```

Output:

```
exouser@node-master:~$ hdfs dfs -cat /output_stat2/part-00000
3:00    66.111.54.249   38
3:00    5.211.97.39     36
3:00    66.249.66.194   31
3:00    31.56.96.51     22
3:00    5.209.200.218   21
exouser@node-master:~$
```

Conclusion: -

Through these steps, we successfully configured a functional hadoop cluster and developed a MapReduce program to process large log files. Firewall rules were applied to ensure secure communication between nodes, and web-based monitoring was enabled for ease of management. The program efficiently extracted top k IPs within a given hour and a specified time period, demonstrating the power of distributed computing with hadoop. **Additionally, we have shelved the instances before submission.**